

Lab7 ORM介绍和基础查询练习

1. ORM的介绍

ORM: Object Relation Mapping, 最初主要描述的是程序中的Object对象和关系型数据库中Relation关系(表)之间的映射关系, 目前来说也是描述程序中对象和数据库中数据记录之间的映射关系的统称, 是一种进行程序和数据库之间数据持久化的一种编程思想。

特点是操纵Python对象而不是SQL查询, 也就是在代码层面考虑的是对象, 而不是SQL, 体现的是一种程序化思维, 这样使得Python程序更加简洁易读。

增删改操作

常规情况下, 软件程序中的ORM操作主要有四个操作场景: 增、删、改、查. 核心操作一般会区分为: 增删改、查询

增加操作: 程序中存在的一个对象Object数据, 通过[ORM]核心模块进行增加的函数定义将对象保存到数据库的操作过程;

如: 注册操作中, 通过用户输入的账号密码等信息创建了一个独立的对象, 通过 `add()` 函数将对象增加保存到数据库中, 数据库中就存在用户这个对象数据了。

修改操作: 程序中存在的一个对象Object数据, 有自己的id编号(可以是程序中自行赋值定义、更多的操作是从数据库中查询出来存在的一个对象), 通过[ORM]核心模块进行修改函数的定义将对象改变的数据更新到数据库中已经存在的记录中的过程;

如: 用户更改登录密码操作时, 根据程序中查询得到的一个用户[id编号、账号、密码、..], 在程序中通过改变其密码属性数据, 然后通过 `update()` 函数将改变的数据更新保存到数据库中, 数据库中原来的数据就发生了新的改变。

删除操作: 程序中存在的一个对象或者已知的id编号, 通过主键编号或者对象的任意属性进行数据库中数据记录的删除的操作过程;

如: 管理员删除某个会员账号的操作, 通过获取要删除会员的账号, 然后通过 `delete()` 函数将要删除的会员信息告知数据库执行删除操作, 数据库中的某条存在的数据记录就被删除掉了。

2.sqlalchemy

2.1 sqlalchemy的介绍和安装

SQLAlchemy 是一个Python 的SQL 工具包以及数据库对象映射框架。它包含整套企业级持久化模式, 专门为高效和高性能的数据库访问。

如果想在本地安装, 可使用以下语句:

```
pip install SQLAlchemy
pip install psycopg2
```

在该水杉环境中已经安装完成, 直接导入即可:

```
In [2]: import sqlalchemy
```

2.2 sqlalchemy 的简单操作

```
In [3]: from sqlalchemy import Column, String, create_engine, Integer, Text, Date
from sqlalchemy.orm import sessionmaker
from sqlalchemy.ext.declarative import declarative_base
import time
```

2.2.1 建立连接

在网址 `postgresql://ecnu学号:ECNU学号@pgm-uf67ua6nah7b72h99o.rwlb.rds.aliyuncs.com:5432/ecnu学号` 中填入自己的学号

```
In [4]: from sqlalchemy import create_engine
engine = create_engine("postgresql://ecnu10245102456:ECNU10245102456@pgm-uf67ua6nah7b72h99o.rwlb.rds.aliyuncs.com:5432/ecnu10245102456")
echo=True,
pool_size=8,
pool_recycle=60*30
)
```

2.2.2 建立会话 (session)

session 用于创建程序与数据库之间的对话。

```
In [5]: from sqlalchemy.orm import sessionmaker
# 创建session
DbSession = sessionmaker(bind=engine)
session = DbSession()
```

session 的常见用法:

1. commit: 提交了一个事务
2. rollback: 回滚
3. close: 关闭

2.2.3 创建表格

declarative_base()是sqlalchemy内部封装的一个方法，通过其构造一个基类，这个基类和它的子类，可以将Python类和数据库表关联映射起来。

数据库表模型类通过tablename和表关联起来，Column表示数据表的列。

```
In [6]: from sqlalchemy.ext.declarative import declarative_base

# 创建对象的基类:
Base = declarative_base()
```

```
C:\Users\gzs27\AppData\Local\Temp\ipykernel_40688\3927807320.py:4: MovedIn20Warning: The ``declarative_base()`` function is now available as sqlalchemy.orm.declarative_base(). (deprecated since: 2.0) (Background on SQLAlchemy 2.0 at: https://sqlalche.me/e/b8d9)
Base = declarative_base()
```

```
In [7]: # 定义User对象:
class User(Base):
    # 表的名称:
    __tablename__ = 'users'
```

```

# 表的结构:
id = Column(Integer, autoincrement=True, primary_key=True, unique=True, nullable=False)
name = Column(String(50), nullable=False)
sex = Column(String(4), nullable=False)
nation = Column(String(20), nullable=False)
birth = Column(String(8), nullable=False)
id_address = Column(Text, nullable=False)
id_number = Column(String(18), nullable=False)
creator = Column(String(32))
create_time = Column(String(20), nullable=False)
updater = Column(String(32))
update_time = Column(String(20), nullable=False,
default=time.strftime("%Y-%m-%d %H:%M:%S", time.localtime()),
onupdate=time.strftime("%Y-%m-%d %H:%M:%S", time.localtime()))
comment = Column(String(200))

def createTable():
    # 创建所有继承于Base的类对应的表
    Base.metadata.create_all(engine)

createTable()

```

```

2026-04-30 08:06:50,793 INFO sqlalchemy.engine.Engine select pg_catalog.version()
2026-04-30 08:06:50,794 INFO sqlalchemy.engine.Engine [raw sql] {}
2026-04-30 08:06:50,814 INFO sqlalchemy.engine.Engine select current_schema()
2026-04-30 08:06:50,814 INFO sqlalchemy.engine.Engine [raw sql] {}
2026-04-30 08:06:50,827 INFO sqlalchemy.engine.Engine show standard_conforming_strings
2026-04-30 08:06:50,829 INFO sqlalchemy.engine.Engine [raw sql] {}
2026-04-30 08:06:50,842 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-04-30 08:06:50,849 INFO sqlalchemy.engine.Engine SELECT pg_catalog.pg_class.relname
FROM pg_catalog.pg_class JOIN pg_catalog.pg_namespace ON pg_catalog.pg_namespace.oid = pg_cata
log.pg_class.relnamespace
WHERE pg_catalog.pg_class.relname = %(table_name)s AND pg_catalog.pg_class.relkind = ANY (ARRA
Y[%(param_1)s, %(param_2)s, %(param_3)s, %(param_4)s, %(param_5)s]) AND pg_catalog.pg_table_is
_visible(pg_catalog.pg_class.oid) AND pg_catalog.pg_namespace.nspname != %(nspname_1)s
2026-04-30 08:06:50,850 INFO sqlalchemy.engine.Engine [generated in 0.00127s] {'table_name':
'users', 'param_1': 'r', 'param_2': 'p', 'param_3': 'f', 'param_4': 'v', 'param_5': 'm', 'nspn
ame_1': 'pg_catalog'}
2026-04-30 08:06:50,870 INFO sqlalchemy.engine.Engine
CREATE TABLE users (
    id SERIAL NOT NULL,
    name VARCHAR(50) NOT NULL,
    sex VARCHAR(4) NOT NULL,
    nation VARCHAR(20) NOT NULL,
    birth VARCHAR(8) NOT NULL,
    id_address TEXT NOT NULL,
    id_number VARCHAR(18) NOT NULL,
    creator VARCHAR(32),
    create_time VARCHAR(20) NOT NULL,
    updater VARCHAR(32),
    update_time VARCHAR(20) NOT NULL,
    comment VARCHAR(200),
    PRIMARY KEY (id),
    UNIQUE (id)
)

2026-04-30 08:06:50,871 INFO sqlalchemy.engine.Engine [no key 0.00140s] {}
2026-04-30 08:06:50,898 INFO sqlalchemy.engine.Engine COMMIT

```

2.2.4 插入数据

```

In [8]: # 插入操作
def insertData():

```

```

# 创建会话
session = DbSession()

# 创建新用户对象:
local_time = time.strftime("%Y-%m-%d %H:%M:%S", time.localtime())
new_user = User(name='mdotdot', sex='女', nation='汉',
birth='19981021', id_address='ECNU', id_number='441242142142',
create_time=local_time)
new_user1 = User(name='xdot', sex='男', nation='汉',
birth='19990110', id_address='ECNU', id_number='451242142142',
create_time=local_time)

# 添加到session:
session.add(new_user)
session.add(new_user1)
# 提交即保存到数据库:
session.commit()

# 关闭session:
session.close()

insertData()

```

```

2026-04-30 08:06:58,385 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-04-30 08:06:58,389 INFO sqlalchemy.engine.Engine INSERT INTO users (name, sex, nation, birth, id_address, id_number, creator, create_time, updater, update_time, comment) SELECT p0::VARCHAR, p1::VARCHAR, p2::VARCHAR, p3::VARCHAR, p4::TEXT, p5::VARCHAR, p6::VARCHAR, p7::VARCHAR, p8::VARCHAR, p9::VARC ... 426 characters truncated ... p4, p5, p6, p7, p8, p9, p10, sen_counter) ORDER BY sen_counter RETURNING users.id, users.id AS id__1
2026-04-30 08:06:58,390 INFO sqlalchemy.engine.Engine [generated in 0.00011s (insertmanyvalues) 1/1 (ordered)] {'creator__0': None, 'nation__0': '汉', 'update_time__0': '2026-04-30 08:06:41', 'id_number__0': '441242142142', 'birth__0': '19981021', 'sex__0': '女', 'id_address__0': 'ECNU', 'create_time__0': '2026-04-30 08:06:58', 'comment__0': None, 'name__0': 'mdotdot', 'updater__0': None, 'creator__1': None, 'nation__1': '汉', 'update_time__1': '2026-04-30 08:06:41', 'id_number__1': '451242142142', 'birth__1': '19990110', 'sex__1': '男', 'id_address__1': 'ECNU', 'create_time__1': '2026-04-30 08:06:58', 'comment__1': None, 'name__1': 'xdot', 'updater__1': None}
2026-04-30 08:06:58,403 INFO sqlalchemy.engine.Engine COMMIT

```

2.2.5 查询数据

SQL 与 SQLAlchemy 的写法区别为:

- query : 对应 SELECT xxx FROM xxx
- filter/filter_by : 对应 WHERE , filter 可以进行比较运算(=, >, < ...)来对条件进行灵活的运用, 不同的条件用逗号分割, filter_by 只能指定参数传参来获取查询结果。
- limit : 对应 limit()
- order by : 对应 order_by()
- group by : 对应 group_by()

返回结果数量可以有以下两种方式:

all()

- 查询所有
- 返回一个列表对象

first()

- 查询第一个符合条件的对象
- 返回一个对象

在ORM中，查询也有和SQL类似的关键字

```
In [9]: from sqlalchemy import and_,or_
```

like	session.query(Person).filter(Person.desc.like("活%")).all()
not like	session.query(Person).filter(Person.desc.notlike("活%")).all()
is(等价于 ==)	session.query(Person).filter(Person.username.is_(None)).all(), session.query(Person).filter(Person.username == None).all()
isnot(等价于 !=)	session.query(Person).filter(Person.username.isnot(None)).all(), session.query(Person).filter(Person.username != None).all()
正则查询	session.query(Person).filter(Person.password.op("regexp")(r"^\u4e00-\u9fa5+")).all()
count	session.query(Person).filter(Person.desc.like("活%")).count()
in	session.query(Person).filter(Person.username.in_(['Mark', 'Tony'])).all()
not in(等价于 ~in)	session.query(Person).filter(Person.username.notin_(['Mark', 'Tony'])).all(), session.query(Person).filter(~Person.username.in_(['Mark', 'Tony'])).all()
AND(导入 and_)	more_person = session.query(Person).filter(and_(Person.password=='123456',Person.desc=="可爱")).all()
OR(导入 or_)	session.query(Person).filter(or_(Person.password=='123456',Person.desc=="活泼")).all()
limit	session.query(Person).filter(Person.desc.notlike("活%")).limit(1).all()
offset	session.query(Person).filter(Person.desc.like("活%")).offset(1).all()
order_by (asc正序)	session.query(Person).order_by(Person.username.desc()).all()
group_by	session.query(Person).group_by(Person.desc).all()
between	session.query(Protocols.protocolName).filter(Protocols.id.between(1, 3)).all()

聚合函数

```
In [10]: from sqlalchemy import func, extract
```

关键字	示例
count	session.query(Person.password, func.count(Person.id)).group_by(Person.password).all()
sum	session.query(Person.password, func.sum(Person.id)).group_by(Person.password).all()
max	session.query(Person.password, func.max(Person.id)).group_by(Person.password).all()
min	session.query(Person.password, func.min(Person.id)).group_by(Person.password).all()
having	session.query(Person.password, func.count(Person.id)).group_by(Person.password).having(func.count(Person.id) > 1).all()

```
In [11]: #查询所有数据
def find_all():
    # 创建Session
    session = DbSession()

    user = session.query(User).all()
    for i in user:
        print('id:', i.id)
        print('name:', i.name)
        print('id_address:', i.id_address)
```

```

        print('id_number:', i.id_number)

    session.close() # 关闭Session

find_all()

```

```

2026-04-30 08:07:48,710 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-04-30 08:07:48,714 INFO sqlalchemy.engine.Engine SELECT users.id AS users_id, users.name AS users_name, users.sex AS users_sex, users.nation AS users_nation, users.birth AS users_birt
h, users.id_address AS users_id_address, users.id_number AS users_id_number, users.creator AS users_creator, users.create_time AS users_create_time, users.updater AS users_updater, users.u
pdate_time AS users_update_time, users.comment AS users_comment
FROM users
2026-04-30 08:07:48,715 INFO sqlalchemy.engine.Engine [generated in 0.00102s] {}
id: 1
name: mdotdot
id_address: ECNU
id_number: 441242142142
id: 2
name: xdot
id_address: ECNU
id_number: 451242142142
2026-04-30 08:07:48,730 INFO sqlalchemy.engine.Engine ROLLBACK

```

```

In [12]: # 查询操作
def selectData():
    # 创建Session
    session = DbSession()

    # 创建Query查询, filter是where条件, 最后调用one()返回唯一行, 如果调用all()则返回所有行:
    user = session.query(User).filter(User.id == '1' and User.name == 'mdotdot').first()

    if user:
        print('name:', user.name)
        print('id_address:', user.id_address)

    session.close() # 关闭Session

selectData()

```

```

2026-04-30 08:07:51,027 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-04-30 08:07:51,030 INFO sqlalchemy.engine.Engine SELECT users.id AS users_id, users.name AS users_name, users.sex AS users_sex, users.nation AS users_nation, users.birth AS users_birt
h, users.id_address AS users_id_address, users.id_number AS users_id_number, users.creator AS users_creator, users.create_time AS users_create_time, users.updater AS users_updater, users.u
pdate_time AS users_update_time, users.comment AS users_comment
FROM users
WHERE users.id = %(id_1)s
LIMIT %(param_1)s
2026-04-30 08:07:51,031 INFO sqlalchemy.engine.Engine [generated in 0.00109s] {'id_1': '1', 'p
aram_1': 1}
name: mdotdot
id_address: ECNU
2026-04-30 08:07:51,045 INFO sqlalchemy.engine.Engine ROLLBACK

```

- 还可以将查询的参数单独写:

```

In [13]: # 创建Session
session = DbSession()

filter = (User.name=='mdotdot')
user = session.query(User).filter(filter).first()
print(user.name)

session.close()

```

```

2026-04-30 08:07:58,994 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-04-30 08:07:58,997 INFO sqlalchemy.engine.Engine SELECT users.id AS users_id, users.name
AS users_name, users.sex AS users_sex, users.nation AS users_nation, users.birth AS users_birt
h, users.id_address AS users_id_address, users.id_number AS users_id_number, users.creator AS
users_creator, users.create_time AS users_create_time, users.updater AS users_updater, users.u
pdate_time AS users_update_time, users.comment AS users_comment
FROM users
WHERE users.name = %(name_1)s
LIMIT %(param_1)s
2026-04-30 08:07:58,998 INFO sqlalchemy.engine.Engine [generated in 0.00118s] {'name_1': 'mdot
dot', 'param_1': 1}
mdotdot
2026-04-30 08:07:59,012 INFO sqlalchemy.engine.Engine ROLLBACK

```

2.2.6 修改数据

- 适用于批量修改

```

In [14]: session.query(User).filter_by(name = "mdotdot").update({'name':"Jack"})
         session.commit() # 提交即保存到数据库

```

```

2026-04-30 08:08:01,601 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-04-30 08:08:01,603 INFO sqlalchemy.engine.Engine UPDATE users SET name=%(name)s, update_t
ime=%(update_time)s WHERE users.name = %(name_1)s
2026-04-30 08:08:01,604 INFO sqlalchemy.engine.Engine [generated in 0.00126s] {'name': 'Jack',
'update_time': '2026-04-30 08:06:41', 'name_1': 'mdotdot'}
2026-04-30 08:08:01,618 INFO sqlalchemy.engine.Engine COMMIT

```

```

In [15]: find_all()

```

```

2026-04-30 08:08:04,828 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-04-30 08:08:04,829 INFO sqlalchemy.engine.Engine SELECT users.id AS users_id, users.name
AS users_name, users.sex AS users_sex, users.nation AS users_nation, users.birth AS users_birt
h, users.id_address AS users_id_address, users.id_number AS users_id_number, users.creator AS
users_creator, users.create_time AS users_create_time, users.updater AS users_updater, users.u
pdate_time AS users_update_time, users.comment AS users_comment
FROM users
2026-04-30 08:08:04,830 INFO sqlalchemy.engine.Engine [cached since 16.12s ago] {}
id: 2
name: xdot
id_address: ECNU
id_number: 451242142142
id: 1
name: Jack
id_address: ECNU
id_number: 441242142142
2026-04-30 08:08:04,842 INFO sqlalchemy.engine.Engine ROLLBACK

```

修改成功

- 适用于获取对象的值,进行操作之后修改

```

In [16]: # 更新操作
def updateData():
    session = DbSession() # 创建会话

    users = session.query(User).filter(User.name=="Jack").first()# 查询条件

    if users:
        users.id_number = "abcd" # 更新操作
        session.add(users) # 添加到会话
        session.commit() # 提交即保存到数据库

```

```
session.close() # 关闭会话
```

```
updateData()
```

```
2026-04-30 08:08:07,621 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-04-30 08:08:07,623 INFO sqlalchemy.engine.Engine SELECT users.id AS users_id, users.name
AS users_name, users.sex AS users_sex, users.nation AS users_nation, users.birth AS users_birt
h, users.id_address AS users_id_address, users.id_number AS users_id_number, users.creator AS
users_creator, users.create_time AS users_create_time, users.updater AS users_updater, users.u
pdate_time AS users_update_time, users.comment AS users_comment
FROM users
WHERE users.name = %(name_1)s
LIMIT %(param_1)s
2026-04-30 08:08:07,624 INFO sqlalchemy.engine.Engine [cached since 8.628s ago] {'name_1': 'Ja
ck', 'param_1': 1}
2026-04-30 08:08:07,647 INFO sqlalchemy.engine.Engine UPDATE users SET id_number=%(id_number)
s, update_time=%(update_time)s WHERE users.id = %(users_id)s
2026-04-30 08:08:07,649 INFO sqlalchemy.engine.Engine [generated in 0.00158s] {'id_number': 'a
bcd', 'update_time': '2026-04-30 08:06:41', 'users_id': 1}
2026-04-30 08:08:07,655 INFO sqlalchemy.engine.Engine COMMIT
```

```
In [17]: find_all()
```

```
2026-04-30 08:08:10,105 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-04-30 08:08:10,106 INFO sqlalchemy.engine.Engine SELECT users.id AS users_id, users.name
AS users_name, users.sex AS users_sex, users.nation AS users_nation, users.birth AS users_birt
h, users.id_address AS users_id_address, users.id_number AS users_id_number, users.creator AS
users_creator, users.create_time AS users_create_time, users.updater AS users_updater, users.u
pdate_time AS users_update_time, users.comment AS users_comment
FROM users
2026-04-30 08:08:10,107 INFO sqlalchemy.engine.Engine [cached since 21.39s ago] {}
id: 2
name: xdot
id_address: ECNU
id_number: 451242142142
id: 1
name: Jack
id_address: ECNU
id_number: abcd
2026-04-30 08:08:10,120 INFO sqlalchemy.engine.Engine ROLLBACK
```

已修改成功

2.2.7 删除数据

- 直接将删除语句写成一行

```
In [18]: delete_query = session.query(User).filter(User.name=='xdot').delete()
session.commit() # 提交会话
```

```
2026-04-30 08:08:18,563 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-04-30 08:08:18,566 INFO sqlalchemy.engine.Engine DELETE FROM users WHERE users.name = %(n
ame_1)s
2026-04-30 08:08:18,568 INFO sqlalchemy.engine.Engine [generated in 0.00199s] {'name_1': 'xd
ot'}
2026-04-30 08:08:18,586 INFO sqlalchemy.engine.Engine COMMIT
```

```
In [19]: find_all()
```



```

2026-04-30 08:08:21,405 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-04-30 08:08:21,406 INFO sqlalchemy.engine.Engine SELECT users.id AS users_id, users.name
AS users_name, users.sex AS users_sex, users.nation AS users_nation, users.birth AS users_birt
h, users.id_address AS users_id_address, users.id_number AS users_id_number, users.creator AS
users_creator, users.create_time AS users_create_time, users.updater AS users_updater, users.u
pdate_time AS users_update_time, users.comment AS users_comment
FROM users
2026-04-30 08:08:21,406 INFO sqlalchemy.engine.Engine [cached since 32.69s ago] {}
id: 1
name: Jack
id_address: ECNU
id_number: abcd
2026-04-30 08:08:21,423 INFO sqlalchemy.engine.Engine ROLLBACK

```

- 查找到数据后再删除

```

In [20]: # 删除操作
def deleteData():
    session = DbSession() # 创建会话

    delete_users = session.query(User).filter(User.id == "1").first()
    if delete_users:
        session.delete(delete_users)
        session.commit()

    session.close() # 关闭会话

deleteData()

```

```

2026-04-30 08:08:24,121 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-04-30 08:08:24,123 INFO sqlalchemy.engine.Engine SELECT users.id AS users_id, users.name
AS users_name, users.sex AS users_sex, users.nation AS users_nation, users.birth AS users_birt
h, users.id_address AS users_id_address, users.id_number AS users_id_number, users.creator AS
users_creator, users.create_time AS users_create_time, users.updater AS users_updater, users.u
pdate_time AS users_update_time, users.comment AS users_comment
FROM users
WHERE users.id = %(id_1)s
LIMIT %(param_1)s
2026-04-30 08:08:24,124 INFO sqlalchemy.engine.Engine [cached since 33.09s ago] {'id_1': '1',
'param_1': 1}
2026-04-30 08:08:24,138 INFO sqlalchemy.engine.Engine DELETE FROM users WHERE users.id = %(id)
s
2026-04-30 08:08:24,139 INFO sqlalchemy.engine.Engine [generated in 0.00102s] {'id': 1}
2026-04-30 08:08:24,145 INFO sqlalchemy.engine.Engine COMMIT

```

```

In [21]: find_all()

```

```

2026-04-30 08:08:27,945 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-04-30 08:08:27,947 INFO sqlalchemy.engine.Engine SELECT users.id AS users_id, users.name
AS users_name, users.sex AS users_sex, users.nation AS users_nation, users.birth AS users_birt
h, users.id_address AS users_id_address, users.id_number AS users_id_number, users.creator AS
users_creator, users.create_time AS users_create_time, users.updater AS users_updater, users.u
pdate_time AS users_update_time, users.comment AS users_comment
FROM users
2026-04-30 08:08:27,948 INFO sqlalchemy.engine.Engine [cached since 39.23s ago] {}
2026-04-30 08:08:27,963 INFO sqlalchemy.engine.Engine ROLLBACK

```

数据都成功删除

2.2.8 删除表格

```

In [22]: def dropTable():
    sql = 'DROP TABLE IF EXISTS users;'
    result = engine.execute(sql)

```

```
dropTable()
```

```
-----
AttributeError                                Traceback (most recent call last)
Cell In[22], line 5
      2     sql = 'DROP TABLE IF EXISTS users;'
      3     result = engine.execute(sql)
----> 5     dropTable()

Cell In[22], line 3, in dropTable()
      1 def dropTable():
      2     sql = 'DROP TABLE IF EXISTS users;'
----> 3     result = engine.execute(sql)

AttributeError: 'Engine' object has no attribute 'execute'
```

删除所有表

```
In [23]: # all tables are deleted
Base.metadata.drop_all(engine)
```

```
2026-04-30 08:09:16,452 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-04-30 08:09:16,453 INFO sqlalchemy.engine.Engine SELECT pg_catalog.pg_class.relname
FROM pg_catalog.pg_class JOIN pg_catalog.pg_namespace ON pg_catalog.pg_namespace.oid = pg_cata
log.pg_class.relnamespace
WHERE pg_catalog.pg_class.relname = %(table_name)s AND pg_catalog.pg_class.relkind = ANY (ARRA
Y[%(param_1)s, %(param_2)s, %(param_3)s, %(param_4)s, %(param_5)s]) AND pg_catalog.pg_table_is
_visible(pg_catalog.pg_class.oid) AND pg_catalog.pg_namespace.nspname != %(nspname_1)s
2026-04-30 08:09:16,454 INFO sqlalchemy.engine.Engine [cached since 145.6s ago] {'table_name':
'users', 'param_1': 'r', 'param_2': 'p', 'param_3': 'f', 'param_4': 'v', 'param_5': 'm', 'nspn
ame_1': 'pg_catalog'}
2026-04-30 08:09:16,469 INFO sqlalchemy.engine.Engine
DROP TABLE users
2026-04-30 08:09:16,469 INFO sqlalchemy.engine.Engine [no key 0.00124s] {}
2026-04-30 08:09:16,484 INFO sqlalchemy.engine.Engine COMMIT
```

3. ORM练习

注意：本次练习中的数据为3.4中数据一次插入的结果。如果不小心多次插入，可使用删除数据或者删除表，再重新插入。

3.1 建立连接

```
In [24]: from sqlalchemy import create_engine
engine = create_engine("postgresql://ecnu10245102456:ECNU10245102456@pgm-uf67ua6nah7b72h99o.rv
    echo=True,
    pool_size=8,
    pool_recycle=60*30
)
```

3.2 建立会话

```
In [25]: from sqlalchemy.orm import sessionmaker
DbSession = sessionmaker(bind=engine)
session = DbSession()
```

3.3 表格创建

四个表格分别是 student, course, teacher, score.


```
create table student(  
s_id varchar(10),  
s_name varchar(20),  
s_age date,  
s_sex varchar(10)  
);
```

```
create table course(  
c_id varchar(10),  
c_name varchar(20),  
t_id varchar(10)  
);
```

```
create table teacher (  
t_id varchar(10),  
t_name varchar(20)  
);
```

```
create table score (  
s_id varchar(10),  
c_id varchar(10),  
score integer );
```

```
In [26]: from sqlalchemy import Column, String, Integer, Date  
from sqlalchemy.ext.declarative import declarative_base
```

```
Base = declarative_base()
```

```
class Student(Base):  
    __tablename__ = 'student'  
    s_id = Column(String(10), primary_key=True)  
    s_name = Column(String(20))  
    s_age = Column(Date)  
    s_sex = Column(String(10))
```

```
class Course(Base):  
    __tablename__ = 'course'  
    c_id = Column(String(10), primary_key=True)  
    c_name = Column(String(20))  
    t_id = Column(String(10))
```

```
class Teacher(Base):  
    __tablename__ = 'teacher'  
    t_id = Column(String(10), primary_key=True)  
    t_name = Column(String(20))
```

```
class Score(Base):  
    __tablename__ = 'score'  
    s_id = Column(String(10), primary_key=True)  
    c_id = Column(String(10), primary_key=True)  
    score = Column(Integer)
```

```
Base.metadata.create_all(engine)
```

```

2026-04-30 08:13:22,898 INFO sqlalchemy.engine.Engine select pg_catalog.version()
2026-04-30 08:13:22,898 INFO sqlalchemy.engine.Engine [raw sql] {}
2026-04-30 08:13:22,912 INFO sqlalchemy.engine.Engine select current_schema()
2026-04-30 08:13:22,913 INFO sqlalchemy.engine.Engine [raw sql] {}
2026-04-30 08:13:22,926 INFO sqlalchemy.engine.Engine show standard_conforming_strings
2026-04-30 08:13:22,927 INFO sqlalchemy.engine.Engine [raw sql] {}
2026-04-30 08:13:22,941 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-04-30 08:13:22,944 INFO sqlalchemy.engine.Engine SELECT pg_catalog.pg_class.relname
FROM pg_catalog.pg_class JOIN pg_catalog.pg_namespace ON pg_catalog.pg_namespace.oid = pg_cata
log.pg_class.relnamespace
WHERE pg_catalog.pg_class.relname = %(table_name)s AND pg_catalog.pg_class.relkind = ANY (ARRA
Y[%(param_1)s, %(param_2)s, %(param_3)s, %(param_4)s, %(param_5)s]) AND pg_catalog.pg_table_is
_visible(pg_catalog.pg_class.oid) AND pg_catalog.pg_namespace.nspname != %(nspname_1)s
2026-04-30 08:13:22,945 INFO sqlalchemy.engine.Engine [generated in 0.00138s] {'table_name':
'student', 'param_1': 'r', 'param_2': 'p', 'param_3': 'f', 'param_4': 'v', 'param_5': 'm', 'ns
pname_1': 'pg_catalog'}
2026-04-30 08:13:22,958 INFO sqlalchemy.engine.Engine SELECT pg_catalog.pg_class.relname
FROM pg_catalog.pg_class JOIN pg_catalog.pg_namespace ON pg_catalog.pg_namespace.oid = pg_cata
log.pg_class.relnamespace
WHERE pg_catalog.pg_class.relname = %(table_name)s AND pg_catalog.pg_class.relkind = ANY (ARRA
Y[%(param_1)s, %(param_2)s, %(param_3)s, %(param_4)s, %(param_5)s]) AND pg_catalog.pg_table_is
_visible(pg_catalog.pg_class.oid) AND pg_catalog.pg_namespace.nspname != %(nspname_1)s
2026-04-30 08:13:22,960 INFO sqlalchemy.engine.Engine [cached since 0.01566s ago] {'table_nam
e': 'course', 'param_1': 'r', 'param_2': 'p', 'param_3': 'f', 'param_4': 'v', 'param_5': 'm',
'nspname_1': 'pg_catalog'}
2026-04-30 08:13:22,968 INFO sqlalchemy.engine.Engine SELECT pg_catalog.pg_class.relname
FROM pg_catalog.pg_class JOIN pg_catalog.pg_namespace ON pg_catalog.pg_namespace.oid = pg_cata
log.pg_class.relnamespace
WHERE pg_catalog.pg_class.relname = %(table_name)s AND pg_catalog.pg_class.relkind = ANY (ARRA
Y[%(param_1)s, %(param_2)s, %(param_3)s, %(param_4)s, %(param_5)s]) AND pg_catalog.pg_table_is
_visible(pg_catalog.pg_class.oid) AND pg_catalog.pg_namespace.nspname != %(nspname_1)s
2026-04-30 08:13:22,969 INFO sqlalchemy.engine.Engine [cached since 0.02462s ago] {'table_nam
e': 'teacher', 'param_1': 'r', 'param_2': 'p', 'param_3': 'f', 'param_4': 'v', 'param_5': 'm',
'nspname_1': 'pg_catalog'}
2026-04-30 08:13:22,976 INFO sqlalchemy.engine.Engine SELECT pg_catalog.pg_class.relname
FROM pg_catalog.pg_class JOIN pg_catalog.pg_namespace ON pg_catalog.pg_namespace.oid = pg_cata
log.pg_class.relnamespace
WHERE pg_catalog.pg_class.relname = %(table_name)s AND pg_catalog.pg_class.relkind = ANY (ARRA
Y[%(param_1)s, %(param_2)s, %(param_3)s, %(param_4)s, %(param_5)s]) AND pg_catalog.pg_table_is
_visible(pg_catalog.pg_class.oid) AND pg_catalog.pg_namespace.nspname != %(nspname_1)s
2026-04-30 08:13:22,977 INFO sqlalchemy.engine.Engine [cached since 0.03339s ago] {'table_nam
e': 'score', 'param_1': 'r', 'param_2': 'p', 'param_3': 'f', 'param_4': 'v', 'param_5': 'm',
'nspname_1': 'pg_catalog'}
2026-04-30 08:13:22,985 INFO sqlalchemy.engine.Engine
CREATE TABLE student (
    s_id VARCHAR(10) NOT NULL,
    s_name VARCHAR(20),
    s_age DATE,
    s_sex VARCHAR(10),
    PRIMARY KEY (s_id)
)

2026-04-30 08:13:22,986 INFO sqlalchemy.engine.Engine [no key 0.00110s] {}
2026-04-30 08:13:23,002 INFO sqlalchemy.engine.Engine
CREATE TABLE course (
    c_id VARCHAR(10) NOT NULL,
    c_name VARCHAR(20),
    t_id VARCHAR(10),
    PRIMARY KEY (c_id)
)

2026-04-30 08:13:23,004 INFO sqlalchemy.engine.Engine [no key 0.00114s] {}

```

```
C:\Users\gzsz27\AppData\Local\Temp\ipykernel_40688\609355957.py:4: MovedIn20Warning: The ``declarative_base()`` function is now available as sqlalchemy.orm.declarative_base(). (deprecated since: 2.0) (Background on SQLAlchemy 2.0 at: https://sqlalche.me/e/b8d9)
```

```
Base = declarative_base()
```

```
2026-04-30 08:13:23,011 INFO sqlalchemy.engine.Engine
```

```
CREATE TABLE teacher (  
    t_id VARCHAR(10) NOT NULL,  
    t_name VARCHAR(20),  
    PRIMARY KEY (t_id)  
)
```

```
2026-04-30 08:13:23,013 INFO sqlalchemy.engine.Engine [no key 0.00113s] {}
```

```
2026-04-30 08:13:23,021 INFO sqlalchemy.engine.Engine
```

```
CREATE TABLE score (  
    s_id VARCHAR(10) NOT NULL,  
    c_id VARCHAR(10) NOT NULL,  
    score INTEGER,  
    PRIMARY KEY (s_id, c_id)  
)
```

```
2026-04-30 08:13:23,022 INFO sqlalchemy.engine.Engine [no key 0.00112s] {}
```

```
2026-04-30 08:13:23,029 INFO sqlalchemy.engine.Engine COMMIT
```

3.4 插入数据

```
insert into student (s_id, s_name, s_age, s_sex)  
values  
( '01' , '赵雷' , '1990-01-01' , '男'),  
( '02' , '钱电' , '1990-12-21' , '男'),  
( '03' , '孙风' , '1990-05-20' , '男'),  
( '04' , '李云' , '1990-08-06' , '男'),  
( '05' , '周梅' , '1991-12-01' , '女'),  
( '06' , '吴兰' , '1992-03-01' , '女'),  
( '07' , '郑竹' , '1989-07-01' , '女'),  
( '08' , '王菊' , '1990-01-20' , '女');
```

```
insert into course (c_id, c_name, t_id)  
values  
( '01' , '语文' , '02'),  
( '02' , '数学' , '01'),  
( '03' , '英语' , '03');
```

```
insert into teacher (t_id, t_name)  
values  
( '01' , '张三'),  
( '02' , '李四'),  
( '03' , '王五');
```

```
insert into score (s_id, c_id, score)  
values  
( '01' , '01' , 80),  
( '01' , '02' , 90),  
( '01' , '03' , 99),  
( '02' , '01' , 70),  
( '02' , '02' , 60),  
( '02' , '03' , 80),  
( '03' , '01' , 80),  
( '03' , '02' , 80),  
( '03' , '03' , 80),
```

```
( '04' , '01' , 50),
( '04' , '02' , 30),
( '04' , '03' , 20),
( '05' , '01' , 76),
( '05' , '02' , 87),
( '06' , '01' , 31),
( '06' , '03' , 34),
( '07' , '02' , 89),
( '07' , '03' , 98);
```

思考：怎样写代码可以批量插入数据

```
In [27]: students = [
    Student(s_id='01', s_name='赵雷', s_age='1990-01-01', s_sex='男'),
    Student(s_id='02', s_name='钱电', s_age='1990-12-21', s_sex='男'),
    Student(s_id='03', s_name='孙风', s_age='1990-05-20', s_sex='男'),
    Student(s_id='04', s_name='李云', s_age='1990-08-06', s_sex='男'),
    Student(s_id='05', s_name='周梅', s_age='1991-12-01', s_sex='女'),
    Student(s_id='06', s_name='吴兰', s_age='1992-03-01', s_sex='女'),
    Student(s_id='07', s_name='郑竹', s_age='1989-07-01', s_sex='女'),
    Student(s_id='08', s_name='王菊', s_age='1990-01-20', s_sex='女')
]
courses = [
    Course(c_id='01', c_name='语文', t_id='02'),
    Course(c_id='02', c_name='数学', t_id='01'),
    Course(c_id='03', c_name='英语', t_id='03')
]
teachers = [
    Teacher(t_id='01', t_name='张三'),
    Teacher(t_id='02', t_name='李四'),
    Teacher(t_id='03', t_name='王五')
]
scores = [
    Score(s_id='01', c_id='01', score=80),
    Score(s_id='01', c_id='02', score=90),
    Score(s_id='01', c_id='03', score=99),
    Score(s_id='02', c_id='01', score=70),
    Score(s_id='02', c_id='02', score=60),
    Score(s_id='02', c_id='03', score=80),
    Score(s_id='03', c_id='01', score=80),
    Score(s_id='03', c_id='02', score=80),
    Score(s_id='03', c_id='03', score=80),
    Score(s_id='04', c_id='01', score=50),
    Score(s_id='04', c_id='02', score=30),
    Score(s_id='04', c_id='03', score=20),
    Score(s_id='05', c_id='01', score=76),
    Score(s_id='05', c_id='02', score=87),
    Score(s_id='06', c_id='01', score=31),
    Score(s_id='06', c_id='03', score=34),
    Score(s_id='07', c_id='02', score=89),
    Score(s_id='07', c_id='03', score=98)
]

session.add_all(students)
session.add_all(courses)
session.add_all(teachers)
session.add_all(scores)
session.commit()
```

```

2026-04-30 08:15:20,277 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-04-30 08:15:20,279 INFO sqlalchemy.engine.Engine INSERT INTO course (c_id, c_name, t_id)
VALUES (%(c_id__0)s, %(c_name__0)s, %(t_id__0)s), (%(c_id__1)s, %(c_name__1)s, %(t_id__1)s),
%(c_id__2)s, %(c_name__2)s, %(t_id__2)s)
2026-04-30 08:15:20,280 INFO sqlalchemy.engine.Engine [generated in 0.00008s (insertmanyvalue
s) 1/1 (unordered)] {'c_name__0': '语文', 'c_id__0': '01', 't_id__0': '02', 'c_name__1': '数
学', 'c_id__1': '02', 't_id__1': '01', 'c_name__2': '英语', 'c_id__2': '03', 't_id__2': '03'}
2026-04-30 08:15:20,295 INFO sqlalchemy.engine.Engine INSERT INTO score (s_id, c_id, score) VA
LUES (%(s_id__0)s, %(c_id__0)s, %(score__0)s), (%(s_id__1)s, %(c_id__1)s, %(score__1)s), (%(s_
id__2)s, %(c_id__2)s, %(score__2)s), (%(s_id__3)s, %(c_id__3)s, %(score__3)s), (%(s_id__4)s, %
(c_id__4)s, %(score__4 ... 473 characters truncated ... ore__15)s), (%(s_id__16)s, %(c_id__16)
s, %(score__16)s), (%(s_id__17)s, %(c_id__17)s, %(score__17)s)
2026-04-30 08:15:20,297 INFO sqlalchemy.engine.Engine [generated in 0.00010s (insertmanyvalue
s) 1/1 (unordered)] {'s_id__0': '01', 'c_id__0': '01', 'score__0': 80, 's_id__1': '01', 'c_id_
__1': '02', 'score__1': 90, 's_id__2': '01', 'c_id__2': '03', 'score__2': 99, 's_id__3': '02',
'c_id__3': '01', 'score__3': 70, 's_id__4': '02', 'c_id__4': '02', 'score__4': 60, 's_id__5':
'02', 'c_id__5': '03', 'score__5': 80, 's_id__6': '03', 'c_id__6': '01', 'score__6': 80, 's_id_
__7': '03', 'c_id__7': '02', 'score__7': 80, 's_id__8': '03', 'c_id__8': '03', 'score__8': 80,
's_id__9': '04', 'c_id__9': '01', 'score__9': 50, 's_id__10': '04', 'c_id__10': '02', 'score_
__10': 30, 's_id__11': '04', 'c_id__11': '03', 'score__11': 20, 's_id__12': '05', 'c_id__12': '0
1', 'score__12': 76, 's_id__13': '05', 'c_id__13': '02', 'score__13': 87, 's_id__14': '06', 'c
_id__14': '01', 'score__14': 31, 's_id__15': '06', 'c_id__15': '03', 'score__15': 34, 's_id__1
6': '07', 'c_id__16': '02', 'score__16': 89, 's_id__17': '07', 'c_id__17': '03', 'score__17':
98}
2026-04-30 08:15:20,304 INFO sqlalchemy.engine.Engine INSERT INTO student (s_id, s_name, s_ag
e, s_sex) VALUES (%(s_id__0)s, %(s_name__0)s, %(s_age__0)s, %(s_sex__0)s), (%(s_id__1)s, %(s_n
ame__1)s, %(s_age__1)s, %(s_sex__1)s), (%(s_id__2)s, %(s_name__2)s, %(s_age__2)s, %(s_sex__2)
s), (%(s_id__3)s, %(s_na ... 168 characters truncated ... %(s_name__6)s, %(s_age__6)s, %(s_sex
__6)s), (%(s_id__7)s, %(s_name__7)s, %(s_age__7)s, %(s_sex__7)s)
2026-04-30 08:15:20,306 INFO sqlalchemy.engine.Engine [generated in 0.00008s (insertmanyvalue
s) 1/1 (unordered)] {'s_name__0': '赵雷', 's_id__0': '01', 's_age__0': '1990-01-01', 's_sex__
0': '男', 's_name__1': '钱电', 's_id__1': '02', 's_age__1': '1990-12-21', 's_sex__1': '男', 's_
_name__2': '孙风', 's_id__2': '03', 's_age__2': '1990-05-20', 's_sex__2': '男', 's_name__3':
'李云', 's_id__3': '04', 's_age__3': '1990-08-06', 's_sex__3': '男', 's_name__4': '周梅', 's_i
d__4': '05', 's_age__4': '1991-12-01', 's_sex__4': '女', 's_name__5': '吴兰', 's_id__5': '06',
's_age__5': '1992-03-01', 's_sex__5': '女', 's_name__6': '郑竹', 's_id__6': '07', 's_age__6':
'1989-07-01', 's_sex__6': '女', 's_name__7': '王菊', 's_id__7': '08', 's_age__7': '1990-01-2
0', 's_sex__7': '女'}
2026-04-30 08:15:20,316 INFO sqlalchemy.engine.Engine INSERT INTO teacher (t_id, t_name) VALUE
S (%(t_id__0)s, %(t_name__0)s), (%(t_id__1)s, %(t_name__1)s), (%(t_id__2)s, %(t_name__2)s)
2026-04-30 08:15:20,318 INFO sqlalchemy.engine.Engine [generated in 0.00011s (insertmanyvalue
s) 1/1 (unordered)] {'t_name__0': '张三', 't_id__0': '01', 't_name__1': '李四', 't_id__1': '0
2', 't_name__2': '王五', 't_id__2': '03'}
2026-04-30 08:15:20,328 INFO sqlalchemy.engine.Engine COMMIT

```

3.5 习题

提示：如果写的代码运行出现断开连接，可以通过下面的语句重新连接。（由于代码的不正确导致的）

```
In [28]: session = DbSession()
```

1.查询学生中的所有女生，并将名字按降序排序

```
In [29]: res1 = session.query(Student).filter(Student.s_sex == '女').order_by(Student.s_name.desc()).a
for r in res1:
    print(f"id: {r.s_id}, name: {r.s_name}")
```



```

2026-04-30 08:15:30,463 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-04-30 08:15:30,465 INFO sqlalchemy.engine.Engine SELECT student.s_id AS student_s_id, stu
dent.s_name AS student_s_name, student.s_age AS student_s_age, student.s_sex AS student_s_sex
FROM student
WHERE student.s_sex = %(s_sex_1)s ORDER BY student.s_name DESC
2026-04-30 08:15:30,465 INFO sqlalchemy.engine.Engine [generated in 0.00077s] {'s_sex_1':
'女'}
id: 07, name: 郑竹
id: 08, name: 王菊
id: 05, name: 周梅
id: 06, name: 吴兰

```

2.查询" 01 "课程中成绩最高的5位同学的id和成绩

```

In [30]: res2 = session.query(Score.s_id, Score.score).filter(Score.c_id == '01').order_by(Score.score
for r in res2:
    print(f"s_id: {r.s_id}, score: {r.score}")

```

```

2026-04-30 08:15:34,936 INFO sqlalchemy.engine.Engine SELECT score.s_id AS score_s_id, score.s
core AS score_score
FROM score
WHERE score.c_id = %(c_id_1)s ORDER BY score.score DESC
LIMIT %(param_1)s
2026-04-30 08:15:34,937 INFO sqlalchemy.engine.Engine [generated in 0.00129s] {'c_id_1': '01',
'param_1': 5}
s_id: 01, score: 80
s_id: 03, score: 80
s_id: 05, score: 76
s_id: 02, score: 70
s_id: 04, score: 50

```

3.查询出生年份在1990年的同学(注意: s_age的类型为date)

```

In [31]: from sqlalchemy import extract
res3 = session.query(Student).filter(extract('year', Student.s_age) == 1990).all()
for r in res3:
    print(f"name: {r.s_name}, age: {r.s_age}")

```

```

2026-04-30 08:15:36,950 INFO sqlalchemy.engine.Engine SELECT student.s_id AS student_s_id, stu
dent.s_name AS student_s_name, student.s_age AS student_s_age, student.s_sex AS student_s_sex
FROM student
WHERE EXTRACT(year FROM student.s_age) = %(param_1)s
2026-04-30 08:15:36,951 INFO sqlalchemy.engine.Engine [generated in 0.00121s] {'param_1': 199
0}
name: 赵雷, age: 1990-01-01
name: 钱电, age: 1990-12-21
name: 孙风, age: 1990-05-20
name: 李云, age: 1990-08-06
name: 王菊, age: 1990-01-20

```

4.查询每位同学一共选择了几门课和总成绩

```

In [32]: from sqlalchemy import func
res4 = session.query(Score.s_id, func.count(Score.c_id), func.sum(Score.score)).group_by(Score
for r in res4:
    print(f"s_id: {r[0]}, count: {r[1]}, sum_score: {r[2]}")

```

```

2026-04-30 08:15:38,866 INFO sqlalchemy.engine.Engine SELECT score.s_id AS score_s_id, count(s
core.c_id) AS count_1, sum(score.score) AS sum_1
FROM score GROUP BY score.s_id
2026-04-30 08:15:38,868 INFO sqlalchemy.engine.Engine [generated in 0.00163s] {}
s_id: 06, count: 2, sum_score: 65
s_id: 03, count: 3, sum_score: 240
s_id: 04, count: 3, sum_score: 100
s_id: 01, count: 3, sum_score: 269
s_id: 02, count: 3, sum_score: 210
s_id: 07, count: 2, sum_score: 187
s_id: 05, count: 2, sum_score: 163

```

5.查询01课程或02课程成绩大于85的同学id

```

In [33]: from sqlalchemy import or_, and_
res5 = session.query(Score.s_id).filter(or_(and_(Score.c_id == '01', Score.score > 85), and_(
for r in res5:
    print(f"s_id: {r.s_id}")

```

```

2026-04-30 08:15:41,039 INFO sqlalchemy.engine.Engine SELECT DISTINCT score.s_id AS score_s_id
FROM score
WHERE score.c_id = %(c_id_1)s AND score.score > %(score_1)s OR score.c_id = %(c_id_2)s AND sco
re.score > %(score_2)s
2026-04-30 08:15:41,040 INFO sqlalchemy.engine.Engine [generated in 0.00163s] {'c_id_1': '01',
'score_1': 85, 'c_id_2': '02', 'score_2': 85}
s_id: 01
s_id: 05
s_id: 07

```